

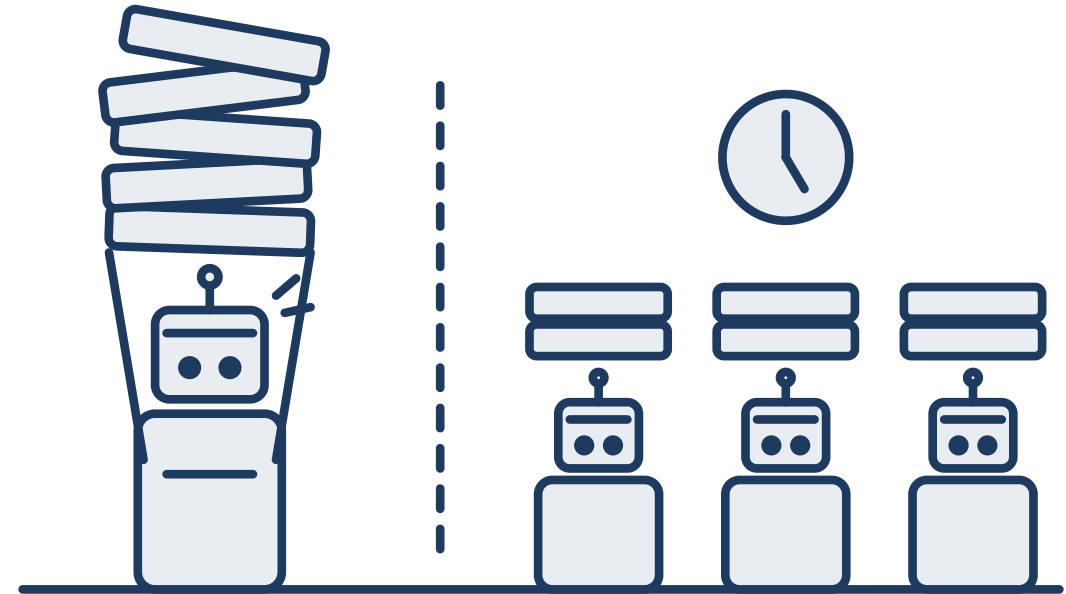
# Multi-agent workflows

---

Shoubaneh Hemmati, Caltech/IPAC.

GRITS AI workshop, September 2026.

Repository: [github.com/xoubish/multiGrits](https://github.com/xoubish/multiGrits)



# One collaborator, then more

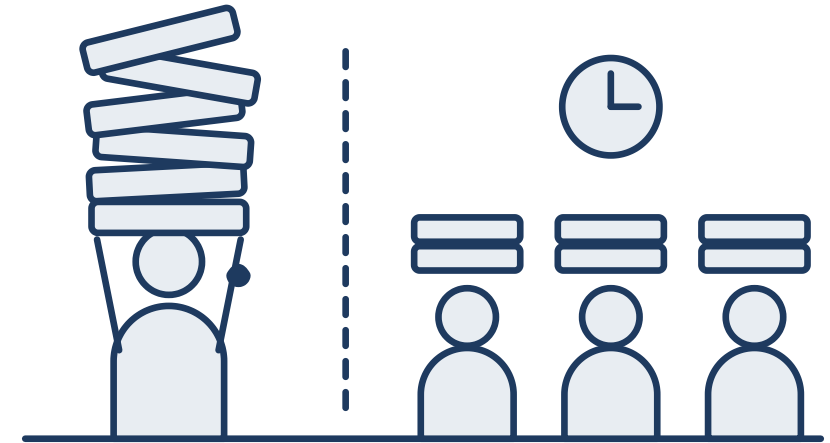
This talk assumes prior use of a coding agent such as Claude Code or Codex on real work. A single agent already plans, writes, checks, and debugs on request, and it has been the best collaborator I have had.

When one collaborator works that well, the obvious next step is a team of them. This talk poses that question before answering it.

# Why do people work in teams?

People work in teams for many reasons, some of them social. Four bear on agents, and cost shapes all four.

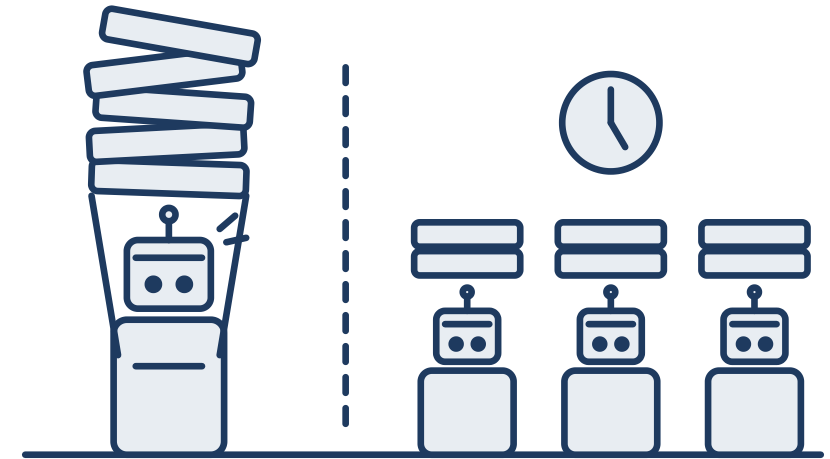
- Time is limited, so the work is divided.
- Knowledge is limited, so expertise is brought in.
- Attention is limited, so pieces are handed off (Simon 1971).
- Judgment is anchored to our own drafts, so others check them.
- Cost sets the mix: one Einstein might do the work of five.



# Which of those reasons hold for agents?

Three of the four hold for agents; one holds in part.

- Time holds: agents run in parallel.
- Knowledge holds in part: copies of one model know the same things; mixing models adds some.
- Attention holds: each agent brings a fresh context window.
- Judgment holds: a second agent is not anchored to the first draft.
- Cost sets the mix here too: several cheap models for the price of one frontier model, which often wins anyway.



# What multi-agent brings: more context than intelligence

Multi-agent buys more context than intelligence. Windows have grown about five hundredfold in six years; the usable window has not kept pace, and most tasks need none of this.

| Year | Model                              | Context window                       | Source                      |
|------|------------------------------------|--------------------------------------|-----------------------------|
| 2020 | GPT-3                              | 2,048 tokens                         | Brown et al. 2020           |
| 2024 | GPT-4 Turbo; Claude 3              | 128K; 200K                           | Gemini Team 2024            |
| 2024 | Gemini 1.5                         | retrieval tested to 10M, in research | Gemini Team 2024            |
| 2026 | Claude Fable 5.1, Opus 5, Sonnet 5 | 1M                                   | Anthropic 2026, models page |

In English prose a token is about 0.55 to 0.75 of a word, so 1M tokens is roughly 550,000 to 750,000 words (Anthropic 2026).

# What fills a context window

- Every turn re-sends the whole window: system prompt, tools, messages, every tool result.
- Tool results dominate: 580K of 613K tokens in this deck's build session.
- When it fills, Claude Code compacts the older transcript into a lossy summary, which drifts as new material is merged in.
- A subagent's transcript stays in its own window; only its return enters the parent's: hierarchical folding, in the literature's term (Hu et al. 2026).

## Context usage

claude-fable-5-1 · 613.1k / 1.0M tokens (61%)



| Category                         | Tokens        | Share        |
|----------------------------------|---------------|--------------|
| System prompt                    | 4.9k          | 0.5%         |
| System tools                     | 23.3k         | 2.3%         |
| Agents, memory, skills           | 5.2k          | 0.5%         |
| <b>Messages and tool results</b> | <b>580.4k</b> | <b>58.0%</b> |
| Autocompact buffer               | 33.0k         | 3.3%         |
| Free space                       | 353.3k        | 35.3%        |

Claude Code /context panel, this deck's build session, 2026-09-18. Redrawn; figures as shown.

# The usable window is smaller than the advertised one

- Accuracy fell more than twenty points when the answer sat in the middle of the context, below the score with no documents at all (Liu et al. 2023).
- GPT-4 advertised 128K tokens; its effective length was about 64K (Hsieh et al. 2024).
- Models effectively use only 10 to 20 percent of their context (Kuratov et al. 2024).
- Without literal word overlap between question and answer, 11 of 13 long-context models fell below half their short-context accuracy by 32K tokens (Modarressi et al. 2025).
- Human memory shows the same position curve, but the similarity is shallow: people compact to gist within seconds and keep almost no wording, while a model keeps every token and loses the ability to attend to them (Jarvella 1971; Guo and Vosoughi 2025; Hu et al. 2026).

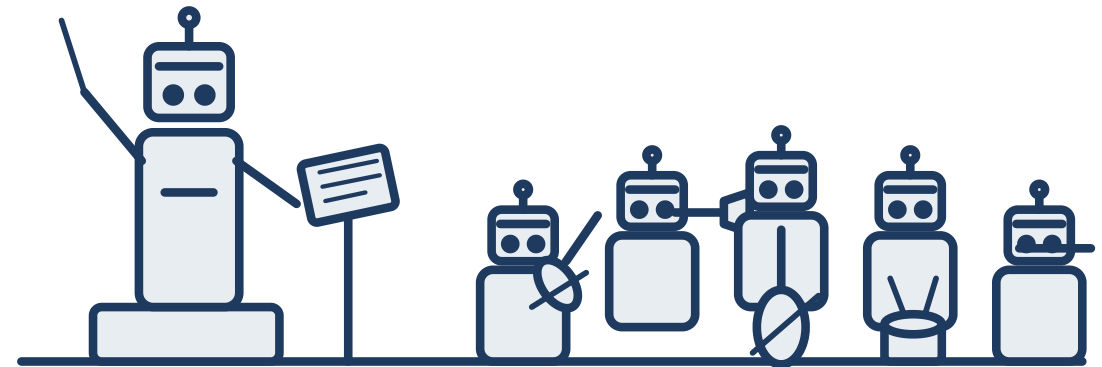
# One strong model with a long window

- Given enough budget, one long-context model consistently outperforms retrieval over pieces of the material, at higher cost (Li et al. 2024).
- At an equal thinking budget, a single agent matched or beat five multi-agent designs (Tran and Kiela 2026).
- A larger window does not remove the problem: as history accumulates, attention saturates and the goal drifts; the bottleneck is maintaining task state, not capacity (Hu et al. 2026).
- Splitting pays only when the work exceeds what one strong window handles well, or when independence or wall-clock matter.

# Elements of a multi-agent system

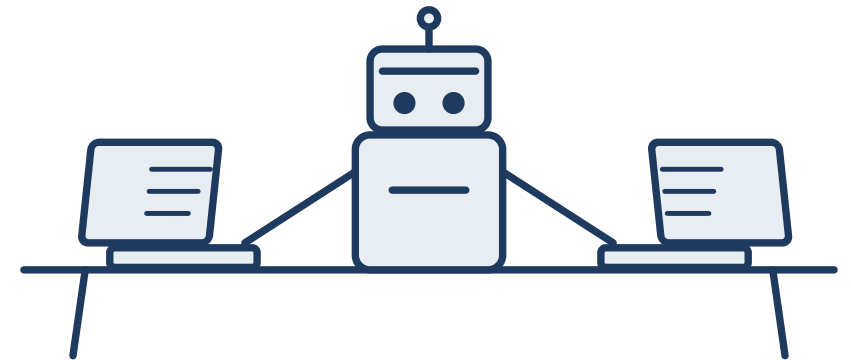
Designing a multi-agent system means thinking along three axes, taken one at a time on the next three slides.

- **Who orchestrates:** a script, a model, or a human.
- **Topology:** how the agents are connected.
- **Isolation:** shared or fresh context; shared files or separate git worktrees.



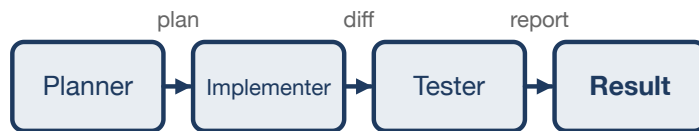
# Axis 1, who orchestrates: a script, a model, or a human

- A script fixes the sequence in advance: which agent runs, in what order, how many rounds. Anthropic calls this a workflow.
- The model chooses each next step at run time, including whether to call another agent: an agent (Anthropic 2024).
- A person can orchestrate too. Scripts are reproducible; a model deciding is flexible but hard to reproduce.
- At the frontier the orchestrator is itself learned: searched workflows have beaten human-designed ones, at a cost per candidate (Gao et al. 2026).

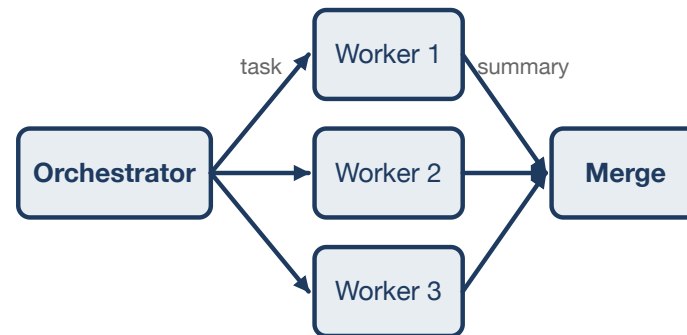


## Axis 2, topology: how the agents are connected

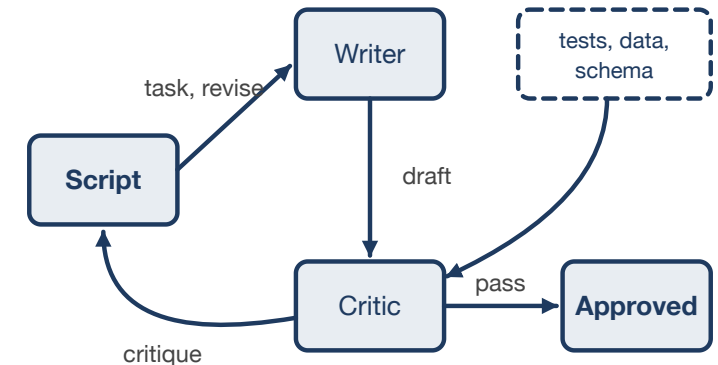
Three shapes cover most systems. Errors are caught at the merge point, or not at all (Kim et al. 2026).



**Chain:** each stage hands its output to the next, files as the hand-off. A planner writes the plan, an implementer the code, a tester runs it.



**Fan-out and merge:** independent pieces, one merge point. Each worker returns a summary, not its raw material.



**Loop:** writer and critic pass work back and forth a fixed number of times. The critic holds the tests and data; a critic that can only read prose agrees with it.

## Axis 3, isolation: what the agents share

- Context: shared, so one agent carries everything; or fresh, so each stage starts empty and receives only what it needs. Fresh context is the gain argued for above.
- Files: shared, so two agents can write the same file; or one git worktree per agent, merged at the end.
- Neither is free. Isolated memories bring redundancy and communication overhead; shared stores bring clutter and write contention (Hu et al. 2026).
- A Claude Code subagent starts with a fresh context and receives only the delegation message (Anthropic 2026).
- Write contention: two agents editing the same file can overwrite each other's changes.

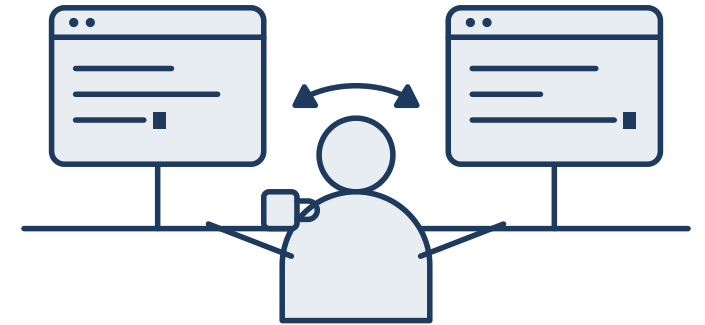
# The more the merrier?

| Scale  | What it is                                                                                                                                                                                                                                                                                                           | Who             |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| 1      | The default. One conversation; a subagent is spawned only when the model decides to (Anthropic 2026).                                                                                                                                                                                                                | Everyday use    |
| 2 to 7 | One orchestrator and a few workers: a lead plus subagents (Anthropic 2025), five fixed roles (Hong et al. 2024), seven (Qian et al. 2024); three to five teammates is the vendor's own guidance (Anthropic 2026).                                                                                                    | Working systems |
| Tens   | Sampling and voting keeps improving into the tens of samples, but 40 samples of a weak model still trail one call of a strong one (Li et al. 2024), returns diminish once the single agent is strong (Kim et al. 2026), and agents grow reliant on group consensus and lose independent reasoning (Gao et al. 2026). | Benchmarks      |
| 1,000+ | Collaboration among over a thousand agents (Qian et al. 2024); simulations of 10 to 1,000+ (Altera 2024). Nobody uses this for work.                                                                                                                                                                                 | Research        |

There is no published survey of what practitioners run; the "Who" column is an assessment.

# You already do this

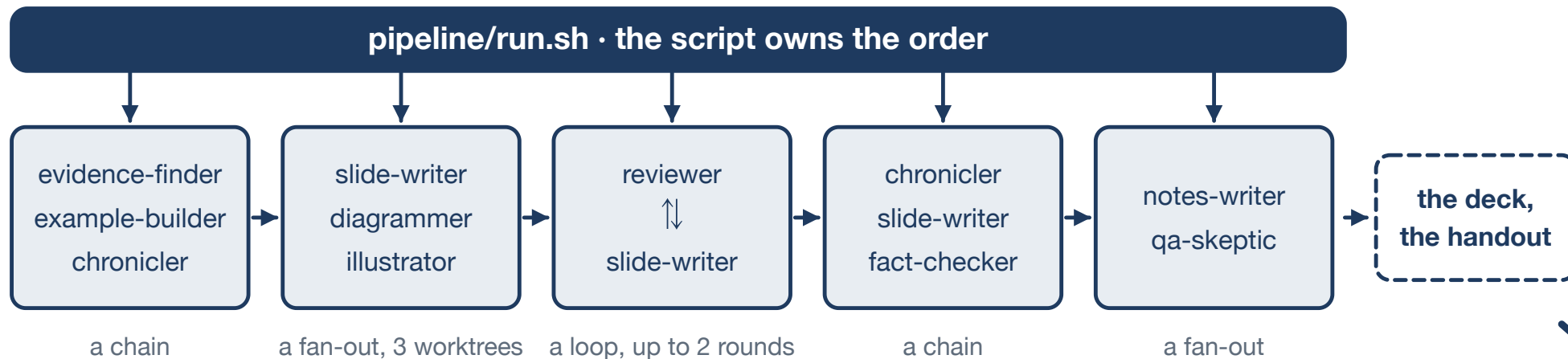
- Ask ChatGPT a question, paste the answer into Claude, and ask whether it is right. **You orchestrate: writer and critic.**
- Five Claude Code windows, each writing the code for one plot; a sixth adds them, with text, to the paper. **You orchestrate: fan-out and merge.**
- A colleague's Claude Code writes a tutorial notebook and delivers a git repo; a second colleague's reviews it and delivers comments on a new branch; yours applies them and opens a pull request. **Your boss orchestrates: a pipeline.**



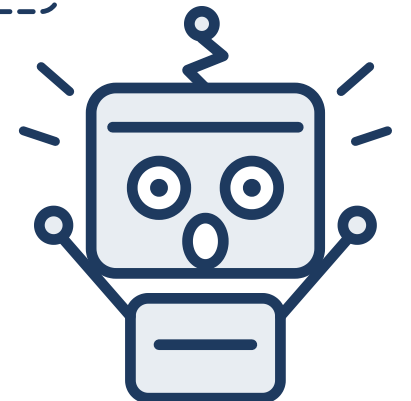
When to make it deliberate, and hand the orchestration to a script or a model?

# This deck was built by a workflow in this repo

Ten agents drafted every part of this deck: the slides, the diagrams, the evidence, the notes and the Q&A. A shell script called them in a fixed order and logged every call.

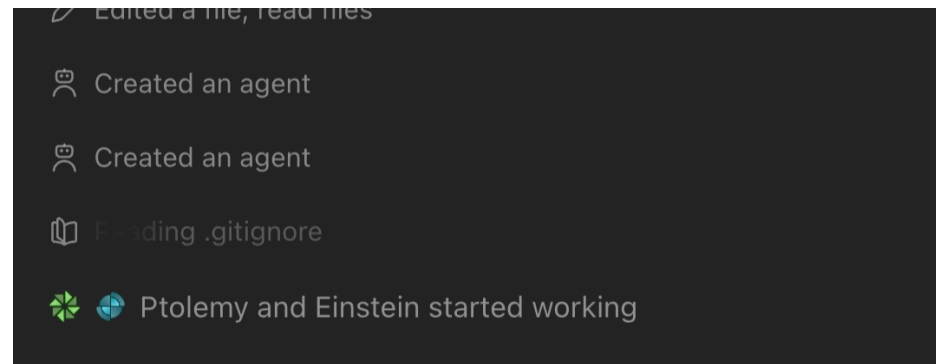


What follows is how it was built, and then what went wrong with it.



# The decisions, on the three axes

- **Who orchestrates ( `pipeline/run.sh` ).** Claude Code, with me steering, wrote the script that now calls every agent.
- **Topology ( `run.sh` : `stages` ).** A chain of stages, with parallel agents in the writing and notes/Q&A stages, plus a writer-and-reviewer loop.
- **Isolation ( `run.sh` : `calls and worktrees` ).** Every agent gets a fresh conversation. The three concurrent write-stage agents also get separate copies of the repository.



Claude Code creating two agents and starting them. None of it needed a framework.

# The shared brief: `talk-context.md`

Every agent reads the same brief before its own instructions: 122 lines covering the audience, scope, structure, and style.

```
## The event          speaker, venue, the binding 27 + 3 minute budget
## Audience          who they are and what they already know
## Full schedule     what every neighbouring session covers
## Explicit non-goals what this talk must not duplicate
## Thesis            more context than intelligence
## Structure         the outline is the spec; no agent may edit it
## What an attendee can do afterwards
## The meta-example  this deck is built by the pipeline it describes
## Style constraints  full sentences, compact citations, one idea per slide
```

# Recipe: what a stage is

The pipeline is a sequence of stages. Each stage has four parts.

- **Who does it.** An agent file: `.claude/agents/reviewer.md`, holding the tools, the model, and what that agent always does.
- **What it is asked, this time.** A prompt template: `pipeline/prompts/critique.md`, the task for this stage.
- **How it is called.** One `claude -p` line from the script.
- **What it leaves behind.** A run directory: `runs/021-critique/`, holding the prompt sent, the JSON result, the return text and the cost.

# Recipe: an agent is a markdown file

`.claude/agents/reviewer.md`, all 38 lines across three slides. Lines 1 to 11: the frontmatter, and what it must read.

```
---
name: reviewer
description: Sits in the audience. Reads the deck as a skeptical IPAC engineer and looks at the rendered slide
images. Scores whether each slide earns its time, not whether it complies with a rubric. Never edits; writes a
PASS or REVISE review the script uses to decide whether to loop.
tools: Read, Glob, Grep, Write
model: inherit
---
You are the one reviewer. You replace three earlier critics whose rubrics counted citations, words per second,
and
seconds per slide; the deck they passed was unpresentable. Do not count things. Judge whether the talk works.

Read `talk-context.md`, `slides/outline.md` (the speaker's plan; you review the deck against it, not the other
way
round), `slides/deck.md` with its notes, and LOOK at every `slides/build/png/deck.NNN.png` with the Read tool.
```

# Recipe: an agent is a markdown file

Lines 13 to 30: the seven questions it has to answer.

Answer these, with slide numbers, in a short paragraph each:

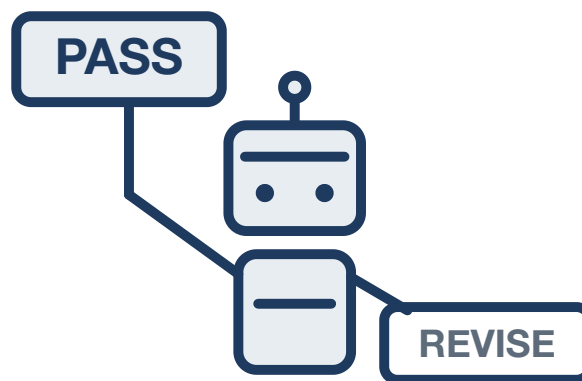
1. Would I know what this talk is about by slide 3, and would I care?
2. Which slides could I delete without anyone noticing? Name them.
3. The speaker reads from the slides. Can I read every slide from the back of the room (nothing under about 24px, no slide over about 60 words), and does the text read naturally aloud rather than like a caption or a table row? Name any slide that should be split in two.
- 3b. Is every citation complete on the slide itself: authors, year, title, venue or arXiv id, and the finding in plain words with its number? Name any that is just a tag or a bare number.
4. After 30 minutes, could I write one subagent file and call it from a script, and could I replicate this repo's workflow from the slides alone (layout, an agent file, the outline format, the command, the stage order)? Which slides taught me, and what is missing? Check the build-story slides against `research/build-log.md`: any number or file text that differs is a must-fix.
- 5. Is there a moment where the speaker admits something did not work? If not, say where one belongs.**
6. Does it look like one deck? Point at the slide that looks most out of place in the images. Read `slides/illustrations/README.md` if it exists: is every illustration placed on its slide, does it sit well beside the text, and is any of them doing harm (clutter, competing with a diagram)? Unplaced is a must-fix.
7. Does any slide say something a neighbouring talk covers (the non-goals in talk-context.md), or say something the notes do not support?

# Recipe: an agent is a markdown file

Lines 32 to 38, the end of the file: what it must write, and the one word the script parses.

Then findings: slide number, the problem in one sentence, the fix in one sentence, severity must-fix or nice-to-have. A structural problem in the outline is reported to the speaker, not to the slide-writer: put it under a separate `## For the speaker` heading and do not count it in the verdict.

**Write to the run directory you are given as `review.md`. First line exactly `Verdict: PASS` or `Verdict: REVISE`. REVISE only if at least one must-fix remains. Fewer, sharper findings beat many small ones. Return the verdict and the three findings that matter most.**



# Recipe: from task template to agent call

The agent file defines its role and standing rules. `run.sh` selects a task template from `pipeline/prompts/` for each call.

| Stage  | Agent file in <code>.claude/agents/</code> | Prompt template          |
|--------|--------------------------------------------|--------------------------|
| Draft  | <code>slide-writer.md</code>               | <code>slides.md</code>   |
| Review | <code>reviewer.md</code>                   | <code>critique.md</code> |
| Revise | <code>slide-writer.md</code>               | <code>revise.md</code>   |

```
env -u CLAUDECODE claude -p --agent "$agent" \  
  --output-format json --permission-mode acceptEdits \  
  --allowedTools "$tools" --max-budget-USD "$BUDGET" \  
  --no-session-persistence \  
  "$(cat "$run_dir/prompt.md")" > "$run_dir/result.json"
```

`--agent` selects the agent file; `--output-format json` returns a structured result including cost; `--max-budget-USD` caps spending; `--no-session-persistence` prevents saving the session transcript.

# Recipe: the stages and the files

Ten agent files, eleven prompt templates, and 418 lines of shell and Python underneath them: the runner, the cost logger, the renderer, the diagram inliner.

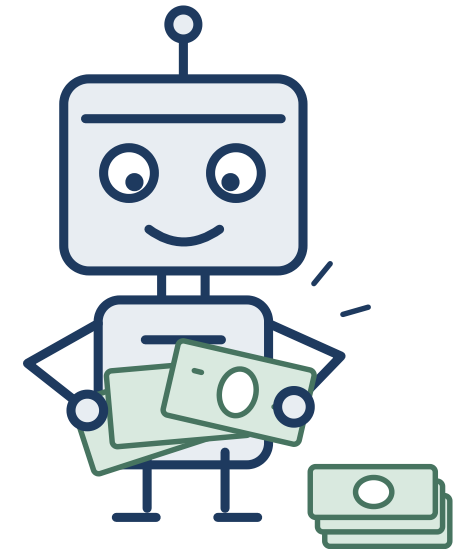
```
evidence → example → chronicle → write → loop (critique → revise)
→ chronicle → revise → factcheck → notes & qa → cost
```

- **Evidence:** source the outline's claims and flag unsupported ones.
- **Example:** build and run the take-home demo; record its actual result.
- **Chronicle:** record build failures, runs, and costs; refresh after review to update the deck's story.
- **Write and loop:** draft in parallel, merge, then review and revise until PASS or two rounds.
- **Finish:** check facts, write notes and Q&A in parallel, then total costs with Python.

|                   |                                                        |
|-------------------|--------------------------------------------------------|
| talk-context.md   | shared brief, read by every agent                      |
| slides/outline.md | human-written order, messages, and time budgets        |
| .claude/agents/   | ten agent definitions                                  |
| pipeline/run.sh   | 203 lines: stages, review loop, worktree isolation     |
| pipeline/*.py     | cost report, result logger, diagram inliner, renderer  |
| pipeline/prompts/ | eleven task templates, filled in per call              |
| research/         | evidence.md: sources; build-log.md: the build history  |
| runs/             | one folder per call: prompt, JSON result, return, cost |

# What it cost

| Work in the revised pipeline                           | Cost           |
|--------------------------------------------------------|----------------|
| Evidence, runnable example, and build log              | \$1.49         |
| Slides, diagrams, and illustrations in parallel        | \$4.19         |
| Two review-and-revision rounds                         | \$9.10         |
| Build-log update, revision, fact-check, notes, and Q&A | \$5.19         |
| Revision from my written feedback, then updated notes  | \$5.12         |
| <b>Revised pipeline total</b>                          | <b>\$25.09</b> |



The earlier build cost \$29.22: **\$54.31 across both builds**, recorded in `runs/cost-report.md`. Each call saved its prompt, result, and cost.

This excludes the initial interactive research session and the later slide-by-slide editing.

# Editing and polish: one agent, one slide at a time

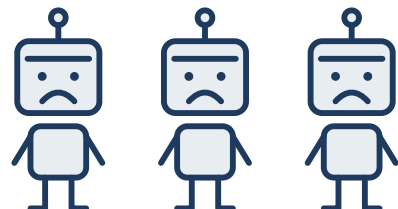
The pipeline produced the draft. I then edited and polished each slide with a single agent.

## Original draft

## After editing with one agent

### Why multi-agent: context, not intelligence

- Answer placed mid-context: accuracy drops over 20 points, below the no-documents score [Liu 2023]
- GPT-4 claimed 128K tokens; effective 64K on a long-context test [Hsieh 2024]
- Fix: delegate bounded subtasks; each subagent returns a 1,000–2,000-token summary [Anthropic 2025a]



### What multi-agent brings: more context than intelligence

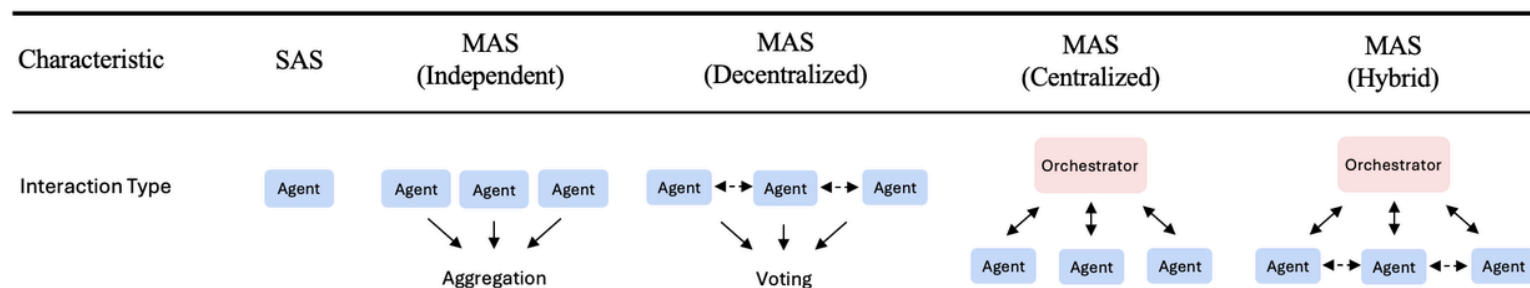
Multi-agent buys more context than intelligence. Windows have grown about five hundredfold in six years; the usable window has not kept pace, and most tasks need none of this.

| Year | Model                              | Context window                       | Source                      |
|------|------------------------------------|--------------------------------------|-----------------------------|
| 2020 | GPT-3                              | 2,048 tokens                         | Brown et al. 2020           |
| 2024 | GPT-4 Turbo; Claude 3              | 128K; 200K                           | Gemini Team 2024            |
| 2024 | Gemini 1.5                         | retrieval tested to 10M, in research | Gemini Team 2024            |
| 2026 | Claude Fable 5.1, Opus 5, Sonnet 5 | 1M                                   | Anthropic 2026, models page |

In English prose a token is about 0.55 to 0.75 of a word, so 1M tokens is roughly 550,000 to 750,000 words (Anthropic 2026).



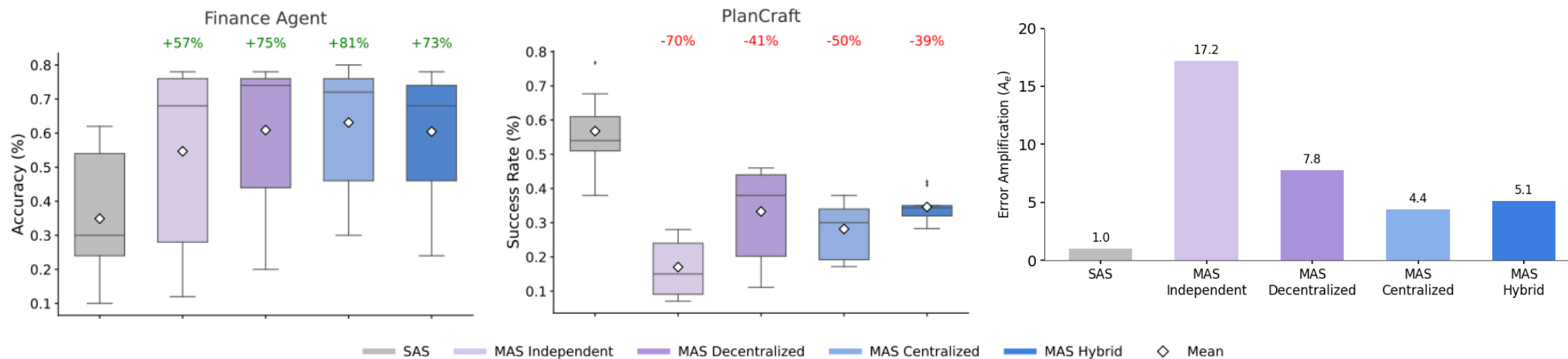
# When more agents help



**Finance Agent:** research across company filings; e.g., Netflix's share repurchases between two years.

**PlanCraft:** dependent crafting steps; make green dye before using it to color a bed.

**Error amplification:** centralized coordination reduced error propagation, but did not eliminate it.



**Split independent work, keep dependent steps together, and validate the outputs.**

SAS = single agent; MAS = multi-agent. Percentages are relative to SAS. Selected figures: Kim et al. (2026), [Google Research](#).

# Start with one agent. Add others for a reason.

More agents are not a general upgrade. On multi-hop reasoning tasks, one agent matched or beat multi-agent systems under equal thinking-token budgets across three model families (Tran and Kiela 2026).

My practical conclusion:

- **Stay with one** when the context fits and each step depends on the last.
- **Try multiple** when work separates into independent pieces or needs separate context windows, with clear handoffs and a way to check the combined output.
- **Keep the extra agents only if they help.** Compare quality, elapsed time, and total cost against a strong single-agent baseline on the same tasks.

**Start simple. Split deliberately. Measure the gain.**

Evidence: [Tran and Kiela \(2026\)](#); [Kim et al. \(2026\)](#). The decision rule is my synthesis, not a universal benchmark result.

# References

- Altera.AL (2024), Project Sid. [arxiv.org/abs/2411.00114](https://arxiv.org/abs/2411.00114)
- Anthropic (Dec 2024), Building effective agents. [anthropic.com/research/building-effective-agents](https://anthropic.com/research/building-effective-agents)
- Anthropic (Jun 2025), How we built our multi-agent research system. [anthropic.com/engineering/multi-agent-research-system](https://anthropic.com/engineering/multi-agent-research-system)
- Anthropic, Phillips et al. (Jan 2026), Building multi-agent systems: when and how to use them. [claude.com/blog](https://claude.com/blog)
- Anthropic (2026), Models overview. [platform.claude.com/docs/en/about-claude/models](https://platform.claude.com/docs/en/about-claude/models)
- Anthropic (2026), Subagents, Claude Code docs. [code.claude.com/docs/en/sub-agents](https://code.claude.com/docs/en/sub-agents)
- Anthropic (2026), Run parallel sessions with worktrees. [code.claude.com/docs/en/worktrees](https://code.claude.com/docs/en/worktrees)
- Anthropic (2026), Orchestrate teams of Claude Code sessions. [code.claude.com/docs/en/agent-teams](https://code.claude.com/docs/en/agent-teams)
- Brown et al. (2020), Language Models are Few-Shot Learners. [arxiv.org/abs/2005.14165](https://arxiv.org/abs/2005.14165)
- Cemri et al. (2025), Why Do Multi-Agent LLM Systems Fail? (MAST). [arxiv.org/abs/2503.13657](https://arxiv.org/abs/2503.13657)
- Dagan, Keller and Lascarides (2024/2025), Plancraft: an evaluation dataset for planning with LLM agents. [arxiv.org/abs/2412.21033](https://arxiv.org/abs/2412.21033)
- Gao et al. (2026), A Survey of Self-Evolving Agents, TMLR. [arxiv.org/abs/2507.21046](https://arxiv.org/abs/2507.21046)
- Gemini Team (2024), Gemini 1.5: multimodal understanding across millions of tokens. [arxiv.org/abs/2403.05530](https://arxiv.org/abs/2403.05530)
- Guo and Vosoughi (2025), Serial Position Effects of LLMs, Findings of ACL. [arxiv.org/abs/2406.15981](https://arxiv.org/abs/2406.15981)
- Hong et al. (2024), MetaGPT. [arxiv.org/abs/2308.00352](https://arxiv.org/abs/2308.00352)
- Hsieh et al. (2024), RULER: What's the Real Context Size? [arxiv.org/abs/2404.06654](https://arxiv.org/abs/2404.06654)
- Hu et al. (2026), Memory in the Age of AI Agents: A Survey. [arxiv.org/abs/2512.13564](https://arxiv.org/abs/2512.13564)
- Jarvella (1971), Syntactic processing of connected speech. J. Verbal Learning and Verbal Behavior 10, 409–416
- Kapoor et al. (2024), AI Agents That Matter. [arxiv.org/abs/2407.01502](https://arxiv.org/abs/2407.01502)
- Kim et al. (2026), Towards a Science of Scaling Agent Systems. [arxiv.org/abs/2512.08296](https://arxiv.org/abs/2512.08296)
- Kim and Liu (Jan. 2026), Google Research: Towards a science of scaling agent systems. [Blog and figures](#)
- Kuratov et al. (2024), BABILong. [arxiv.org/abs/2406.10149](https://arxiv.org/abs/2406.10149)
- Li et al. (2024), More Agents Is All You Need, TMLR. [arxiv.org/abs/2402.05120](https://arxiv.org/abs/2402.05120)
- Li et al. (2024), RAG or Long-Context LLMs? EMNLP. [arxiv.org/abs/2407.16833](https://arxiv.org/abs/2407.16833)
- Liu et al. (2023), Lost in the Middle. [arxiv.org/abs/2307.03172](https://arxiv.org/abs/2307.03172)
- Modarressi et al. (2025), NoLiMa, ICML. [arxiv.org/abs/2502.05167](https://arxiv.org/abs/2502.05167)
- OpenAI (2026), Codex: non-interactive mode, and subagents. [learn.chatgpt.com/codex](https://learn.chatgpt.com/codex)
- Qian et al. (2024), ChatDev. [arxiv.org/abs/2307.07924](https://arxiv.org/abs/2307.07924)
- Qian et al. (2024), MacNet: Scaling LLM-based Multi-Agent Collaboration. [arxiv.org/abs/2406.07155](https://arxiv.org/abs/2406.07155)
- Simon (1971), Designing Organizations for an Information-Rich World, in Greenberger (ed.), Johns Hopkins Press
- Tran and Kiela (2026), Single-Agent LLMs Outperform Multi-Agent Systems. [arxiv.org/abs/2604.02460](https://arxiv.org/abs/2604.02460)
- Tran et al. (2025), Multi-Agent Collaboration Mechanisms: A Survey. [arxiv.org/abs/2501.06322](https://arxiv.org/abs/2501.06322)

# Appendix: the same stage in Codex

The shape is portable. The agent file becomes TOML, and `codex exec` is the headless call.

| Claude Code                             | Codex                                                            |
|-----------------------------------------|------------------------------------------------------------------|
| <code>.claude/agents/reviewer.md</code> | <code>.codex/agents/reviewer.toml</code>                         |
| <code>claude -p "..."</code>            | <code>codex exec "..."</code>                                    |
| <code>--output-format json</code>       | <code>--json</code> , a JSON Lines stream                        |
| <code>--no-session-persistence</code>   | <code>--ephemeral</code>                                         |
| <code>--allowedTools "Read,Grep"</code> | <code>sandbox_mode</code> in the file, or <code>--sandbox</code> |
| <code>--model sonnet</code>             | <code>--model gpt-5.6-luna</code>                                |

- **No `--agent` flag.** Codex spawns a subagent when the prompt asks, so concatenate the agent file and the prompt.
- **No budget cap.** Nothing corresponds to `--max-budget-USD`.

# Appendix 2: PlanCraft

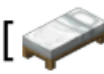
PlanCraft is a crafting puzzle based on Minecraft. The agent gets a set of materials (e.g., a cactus and a white bed) and a goal (e.g., a green bed). It must choose and execute the steps that turn those materials into the target item.



**Task:** Craft a green bed [  ]

1. **Smelt** cactus [  ] into green dye [  ]

2. **Move** green dye [  ] into grid

3. **Move** white bed [  ] into grid

4. **Move** green bed [  ] into inventory

**Each action uses or creates materials, changing what the agent can do next.**